LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 1
MODUL 4
“PROSEDUR”

DISUSUN OLEH:

FARID HERDIYANTO VITASANDI

109082500123

S1 IF-13-02

DOSEN:

Wahyu Andi Saputra, S.pd., M.Eng.

ASISTEN PRAKTIKUM:

Renisa Assyifa Putri

Muhammad Agha Zulfadhli

PROGRAM STUDI S1 TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2024/2025

SOAL LATIHAN

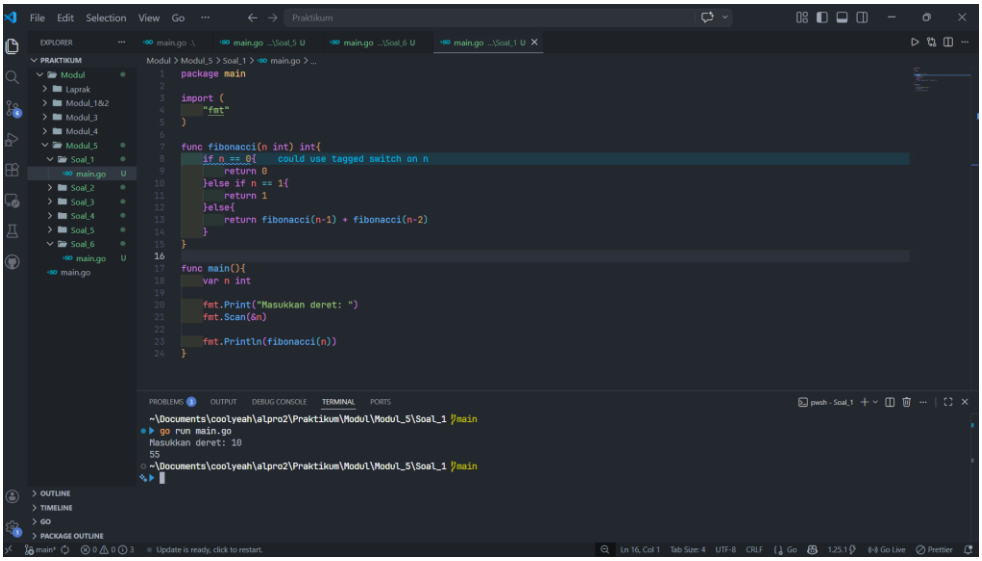
Statement Rekursif

1.

Source Code:

```
package main

import (
	"fmt"
)

func fibonacci(n int) int{
	if n == 0{
		return 0
	}else if n == 1{
		return 1
	}else{
		return fibonacci(n-1) + fibonacci(n-2)
	}
}

func main(){
	var n int

	fmt.Print("Masukkan deret: ")
	fmt.Scan(&n)

	fmt.Println(fibonacci(n))
}
```

Output:



Deskripsi Program:

Kode ini adalah program dalam bahasa Go yang berfungsi untuk menghitung nilai angka pada deret Fibonacci sesuai dengan urutan yang dimasukkan. Inti dari program ini terletak pada fungsi fibonacci yang menggunakan metode rekursi, yaitu sebuah proses di mana fungsi memanggil dirinya sendiri untuk menjumlahkan dua angka sebelumnya (n-1 dan n-2) hingga mencapai kondisi dasar yakni angka 0 atau 1. Di dalam bagian main, program akan meminta user untuk memasukkan sebuah angka melalui terminal, mengirim angka tersebut ke fungsi untuk dihitung secara otomatis, dan kemudian menampilkan hasil akhirnya di layar.

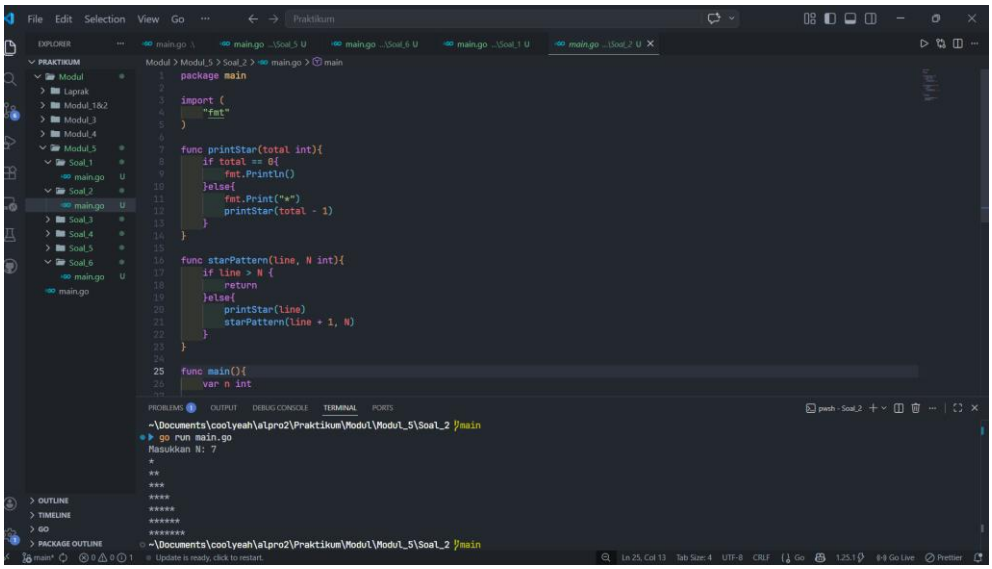
Statement Rekursif

2.

Source Code:

```
package main

import (
	"fmt"
)

func printStar(total int){
	if total == 0{
		fmt.Println()
	}else{
		fmt.Print("*")
		printStar(total - 1)
	}
}

func starPattern(line, N int){
	if line > N {
		return
	}else{
		printStar(line)
		starPattern(line + 1, N)
	}
}

func main(){
	var n int

	fmt.Print("Masukkan N: ")
	fmt.Scan(&n)
	starPattern(1, n)
}
```

Output:



Deskripsi Program:

Kode ini berfungsi untuk membuat pola segitiga bintang menggunakan teknik rekursi, di mana fungsi memanggil dirinya sendiri sebagai pengganti perulangan (looping) biasa. Terdapat dua fungsi utama pada kode ini, printStar yang bertugas mencetak deretan bintang secara horizontal dalam satu baris dan starPattern bertugas mengatur urutan baris dan menentukan kapan proses harus berhenti berdasarkan angka N yang dimasukkan. Secara sederhana, saat program dijalankan, ia akan mulai mencetak satu bintang pada baris pertama dan terus menambah jumlah bintang di setiap baris berikutnya secara otomatis hingga membentuk segitiga siku-siku.

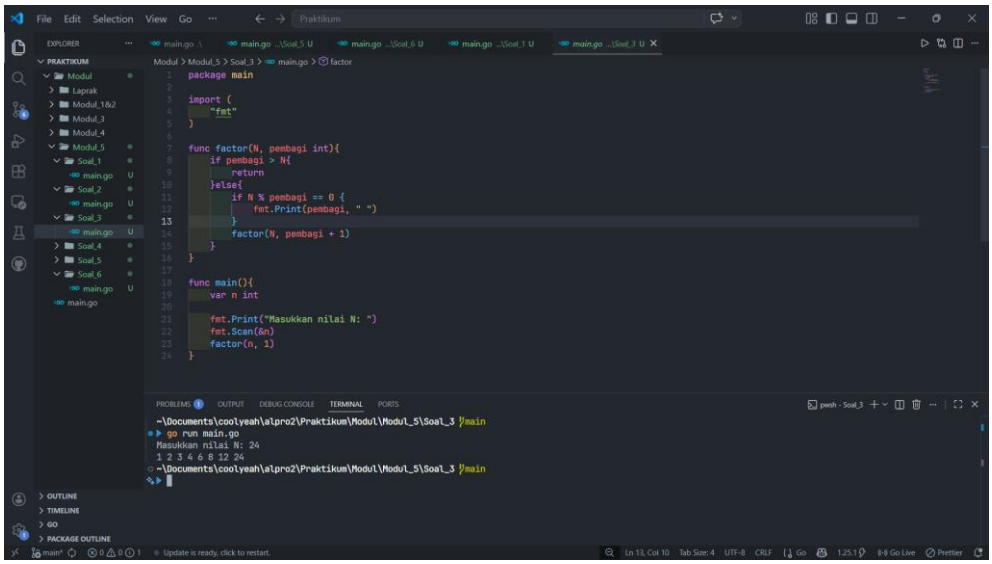
Statement Rekursi

3.

Source Code:

```
package main

import (
	"fmt"
)

func factor(N, pembagi int){
	if pembagi > N{
		return
	}else{
		if N % pembagi == 0 {
			fmt.Print(pembagi, " ")
		}
		factor(N, pembagi + 1)
	}
}

func main(){
	var n int

	fmt.Print("Masukkan nilai N: ")
	fmt.Scan(&n)
	factor(n, 1)
}
```

Output:



Deskripsi Program:

Program ini menggunakan rekursi untuk mencari dan menampilkan semua faktor bilangan dari angka N yang dimasukkan dengan cara mengecek setiap angka satu per satu mulai dari 1. Di dalam fungsi factor, program akan menguji apakah N habis dibagi oleh angka pembagi saat ini, jika benar (sisa bagi adalah nol), angka tersebut dicetak ke layar, lalu fungsi akan memanggil dirinya sendiri secara otomatis dengan menambah nilai pembaginya hingga mencapai batas nilai N.

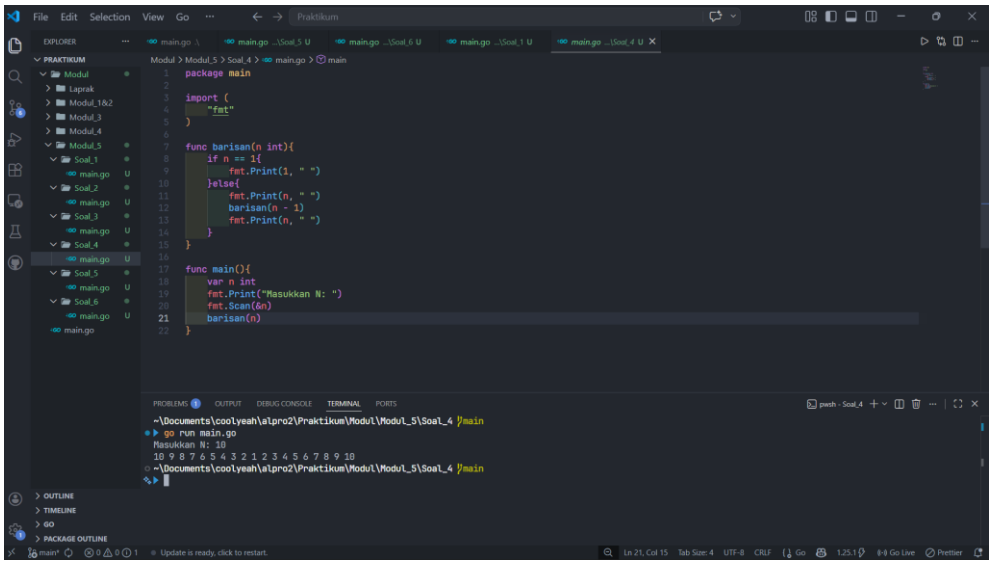
Statement Rekursi

4.

Source Code:

```
package main

import (
	"fmt"
)

func barisan(n int){
	if n == 1{
		fmt.Print(1, " ")
	}else{
		fmt.Print(n, " ")
		barisan(n - 1)
		fmt.Print(n, " ")
	}
}

func main(){
	var n int
	fmt.Print("Masukkan N: ")
	fmt.Scan(&n)
	barisan(n)
}
```

Output:



Deskripsi Program:

Kode ini menggunakan teknik rekursi untuk menciptakan barisan angka simetris yang mencerminkan proses "masuk" dan "keluar" dari sebuah tumpukan fungsi. Saat dijalankan, fungsi barisan akan mencetak angka n saat ini, lalu memanggil dirinya sendiri dengan nilai yang lebih kecil (n-1) hingga mencapai batas akhir yaitu angka 1; setelah mencapai angka 1, program tidak lagi memanggil fungsi baru melainkan menyelesaikan perintah cetak yang tersisa dari panggilan-panggilan sebelumnya. Hal ini menyebabkan angka-angka tercetak menurun hingga angka 1 sebagai pusatnya, kemudian otomatis tercetak kembali secara meningkat, sehingga jika Anda memasukkan angka 3, hasil yang muncul adalah 3 2 1 2 3.

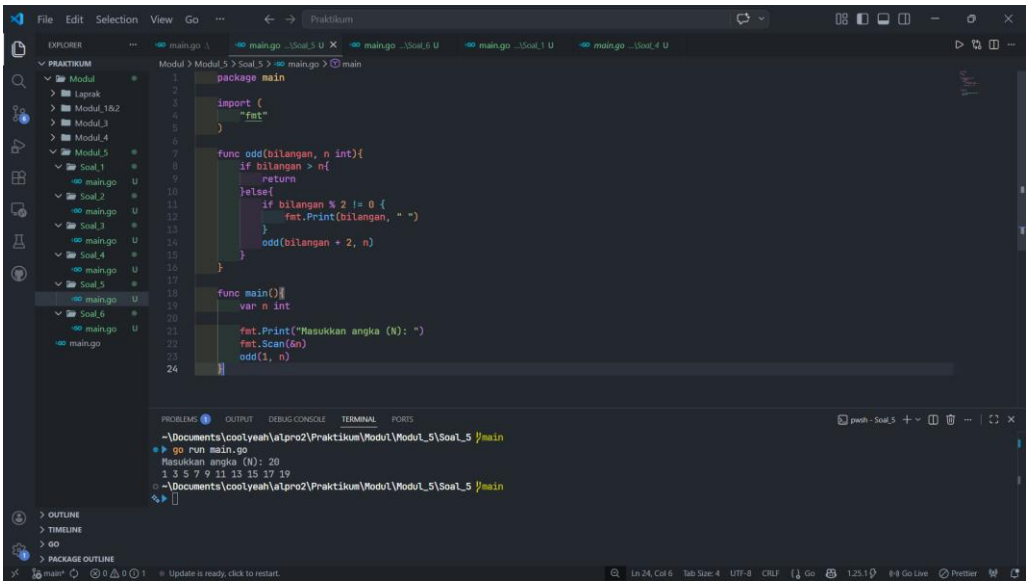
Statement Rekursi

5.

Source Code:

```
package main

import (
    "fmt"
)

func odd(bilangan, n int){
    if bilangan > n{
        return
    }else{
        if bilangan % 2 != 0 {
            fmt.Print(bilangan, " ")
        }
        odd(bilangan + 2, n)
    }
}

func main(){
    var n int

    fmt.Print("Masukkan angka (N): ")
    fmt.Scan(&n)
    odd(1, n)
}
```

Output:



Deskripsi Program:

Program ini menggunakan rekursi untuk menampilkan deret bilangan ganjil dari angka 1 hingga batas n yang ditentukan. Fungsi odd akan mencetak angka ganjil saat ini, lalu memanggil dirinya sendiri dengan menambah nilai angka sebesar 2 hingga mencapai batas n, sehingga berfungsi sebagai cara alternatif pengganti perulangan (looping) konvensional.

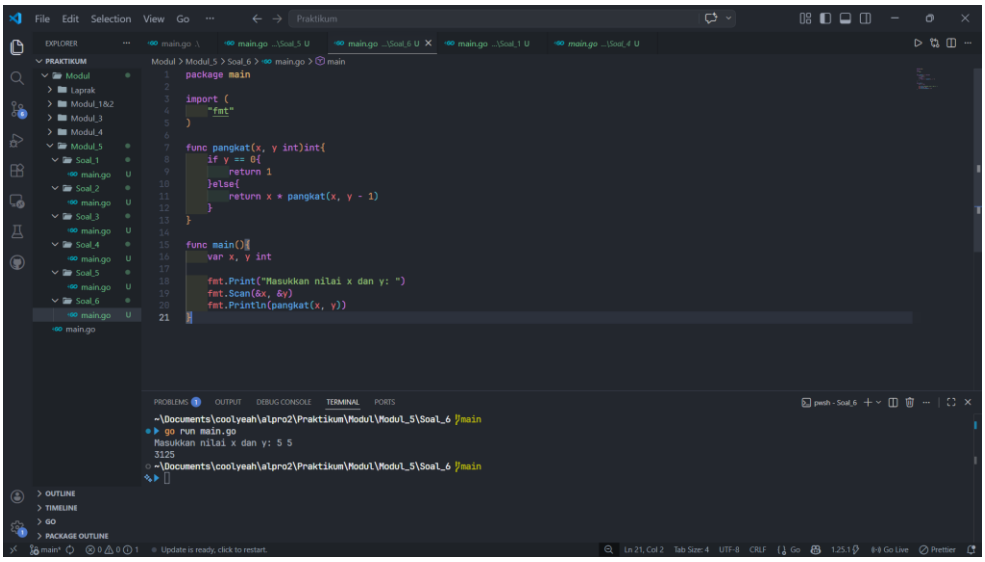
Statement Rekursi

6.

Source Code:

```
package main

import (
	"fmt"
)

func pangkat(x, y int)int{
	if y == 0{
		return 1
	}else{
		return x * pangkat(x, y - 1)
	}
}

func main(){
	var x, y int

	fmt.Print("Masukkan nilai x dan y: ")
	fmt.Scan(&x, &y)
	fmt.Println(pangkat(x, y))
}
```

Output:



Deskripsi Program:

Program ini menggunakan teknik rekursi untuk menghitung hasil perpangkatan x^y dengan cara mengalikan basis x secara berulang sebanyak y kali. Fungsi pangkat akan terus memanggil dirinya sendiri sambil mengurangi nilai eksponen y satu per satu hingga mencapai kondisi berhenti (base case) saat y=0, di mana fungsi akan mengembalikan nilai 1 dan menyelesaikan seluruh rangkaian perkalian yang telah ditumpuk dalam memori.

Daftar Pustaka

[1] Informatics Lab, "*Modul Praktikum Algoritma dan Pemrograman,*" S1 Informatika, Fak. Informatika, Telkom University, Bandung, 2025.